

ELO312 Laboratorio de Estructura de Computadores

Sesión Guiada 1: Programación en C

Rodrigo Jiménez, Mauricio Solís, Alejandro Weinstein
20260812

Esta sesión guiada tiene como objetivo continuar con el proceso de familiarización con los principios básicos del lenguaje de programación C. Para ello, realice las actividades descritas a continuación, procurando escribir el código de cada actividad para sacarle el máximo provecho a esta experiencia (no haga copiar/pegar).

El código de todas las actividades descritas a continuación debe ejecutarlo en la tarjeta Nucleo-L476RG¹, y desarrollarlo en el ambiente de desarrollo STM32CubeIDE,² versión 2.2.0, junto a la herramienta de configuración gráfica STM32CubeMX³, versión 6.18.1. Utilice como punto de partida para todas las actividades de esta sesión el proyecto disponible en https://github.com/aweinstein/elo312_2026, que redirige `stdout`⁴ al puerto serial, lo que permite observar en su computador personal lo que escribe la función `printf`. Instale en su computador personal un terminal serial, como *Termite*⁵, para recibir los caracteres enviados por la tarjeta Nucleo. Note que, si bien los códigos presentados en esta guía no utilizan `printf`, esta función le puede ser útil para entender lo que hace su programa.

Importante: Debe adaptar (ligeramente) los códigos que aparecen más abajo para integrarlos en el proyecto.

1. Arreglos

Considere el siguiente programa:

```
1 int arr1[5] = {1, 2, 3, 4, 5};
2 char arr2[6] = {'E', 'L', 'O', '3', '1', '2'};
3 char arr3[7] = {'2', '1', '3', 'O', 'L', 'E', '\0'};
4 char arr4[7] = "ELO312\0";
5 char arr5[] = "ELO312";
6
7 int main()
8 {
9     int elemento_arr1 = arr1[1];
10    char elemento_arr2 = arr2[3];
11    arr3[0] = 'A';
12    arr4[5] = '3';
13    int arr6[2][2] = {{1,2},{3,4}};
14
15    arr6[0][0] = 5;
16    return 0;
17 }
```

- ¿Qué hace el compilador cuando declaramos un arreglo?
- ¿Cómo se manipulan los arreglos?

¹El código en C lo puede probar, de manera preliminar, usando un compilador C nativo en su computador personal, pero debe ejecutarse en la tarjeta Nucleo para ser considerado válido en el marco de esta sesión de laboratorio.

²Disponible en <https://www.st.com/en/development-tools/stm32cubeide.html>

³Disponible en <https://www.st.com/en/development-tools/stm32cubemx.html>

⁴Ver https://en.wikipedia.org/wiki/Standard_streams para más detalles sobre lo que esto significa.

⁵Disponible en https://www.compuphase.com/software_termite.htm.

- c. ¿Podemos agregar el número 6 a una nueva posición del arreglo `arr1` (manteniendo los demás 5 números) justo antes del `return 0`? ¿Por qué?
- d. ¿Qué sucede con los arreglos `arr2` y `arr3`? ¿Y con `arr4`? ¿Qué son estos arreglos?
- e. ¿Qué puede comentar sobre `arr5`? ¿Cómo se ve la memoria en este caso?
- f. ¿Qué puede comentar sobre `arr6`? ¿Cómo se ve la memoria en este caso?

2. Estructuras de datos

Cree un proyecto nuevo llamado “estructuras” y configúrelo. Luego, agregue un archivo `main.c` con el siguiente código:

```
1 struct _tipo_mascota
2 {
3     char raza[10];
4     char nombre[10];
5     int edad;
6     float peso;
7 };
8
9 struct _tipo_mascota m1;
10
11 int main()
12 {
13     //tipo
14     m1.raza[0] = 'p';
15     m1.raza[1] = 'e';
16     m1.raza[2] = 'r';
17     m1.raza[3] = 'r';
18     m1.raza[4] = 'o';
19     m1.raza[5] = '\0'; //esto es muy tedioso
20     //nombre
21     m1.nombre[0] = 'R';
22     m1.nombre[1] = 'o';
23     m1.nombre[2] = 'c';
24     m1.nombre[3] = '\0';
25     //edad y peso
26     m1.edad = 2;
27     m1.peso = 10.5;
28
29     return 0;
30 }
```

Corra su programa paso a paso, revise las variables en la ventana *expressions* y responda:

- a. ¿Cómo hace el compilador para almacenar la estructura de datos en la memoria?
- b. Reescriba las siguientes líneas, manteniendo el comportamiento del programa :
 - Línea 1: `typedef struct _tipo_mascota`
 - Línea 7: `} tipo_mascota;`
 - Línea 9: `tipo_mascota m1;`

¿Qué es `_tipo_mascota`? ¿y `tipo_mascota`? ¿Qué hace el compilador cuando encuentra la palabra clave `typedef`?

- c. Reescriba el programa de tal forma que no tenga que escribir el nombre y la raza de la mascota carácter por carácter.

3. Uniones

Un caso especial de estructuras de datos son las **uniones**. Estas tienen la particularidad de que todos sus campos ocupan el mismo espacio de memoria. De esta manera, podemos interpretar rápidamente lo que contiene dicha memoria como distintos tipos de datos. Por ejemplo:

```
1 typedef union _tipo_ejemplo
2 {
3     long long largo;    //64 bits
4     int entero[2];      //64 bits
5     unsigned char caracter[8]; //64 bits
6     float flotante[2];  //64 bits
7 } tipo_ejemplo;
8
9 tipo_ejemplo ejemplo;
10
11 int main()
12 {
13     ejemplo.largo = 0x123456789ABCDEF0;
14     return 0;
15 }
```

Cree un nuevo proyecto llamado “union_test”. Configúrelo y corra el ejemplo anterior paso a paso.

- Estudie el concepto de *endianness*⁶.
- Revise en la vista *Memory Browser* del *debug* qué segmento de **memoria** ocupa cada campo de la **union**.
- Contrástelo ahora con los **valores** de dicho campo en la vista de *expressions*.
- En base a lo observado, determine con qué tipo de *endianness* opera STM32 en este caso.
- ¿Qué deberíamos ver en el campo **entero[0]** (en hexadecimal)? ¿Y en **caracter[3]**?
- Si quisiera cambiar el 0x12 por 0xFF, ¿cuál es la forma más rápida de hacerlo?

⁶Lea, por ejemplo, <https://en.wikipedia.org/wiki/Endianness>

4. Punteros

Un puntero es una variable que almacena la dirección de memoria de otra variable. A continuación se explora su ejecución y sintaxis.

Cree un proyecto llamado “punteros” y agregue las siguientes líneas de código:

```
1 typedef struct _estructura
2 {
3     int a;
4     char b;
5 } estructura;
6
7 int main()
8 {
9     int a = 5;
10    int *p1 = &a; // De esta forma se define un puntero a entero
11    *p1 = (*p1) + 1; // que hace esta linea?
12    p1++;          // y esta?
13
14    char nombre[] = "federico";
15    char *p2;
16    p2 = nombre;
17    p2[0] = 'F';
18    char letra = *(nombre + 1);
19    *(p2+7) = 61;
20
21    estructura estr;
22    estructura *pEstr = &estr;
23    (*pEstr).a = 10;
24    pEstr->b = 'k';
25
26    return 0;
27 }
```

Corra este programa paso a paso, observando la ventana *expression*. Responda:

- Explique qué estamos haciendo en la línea 10 al hacer la igualdad.
- ¿Qué y en cuánto se incrementa en la línea 11?
- ¿Qué y en cuánto se incrementa en la línea 12?
- Explique qué estamos haciendo en la línea 16 al establecer la igualdad. ¿Por qué difiere de la línea 10?
- Explique por qué el elemento almacenado en la variable `letra` tiene ese valor.
- ¿Le resulta curioso lo que ocurre en la línea 19? ¿Por qué?
- ¿Qué guarda la variable `pEstr`?
- ¿Qué hacen las líneas 23 y 24?

5. Funciones

En C, las funciones son un segmento de código ejecutable, que valga la redundancia, se ejecutan solo cuando son llamadas. La idea de utilizar funciones es organizar el código de manera más ordenada, ejecutando determinadas acciones referentes al nombre de la función (esto es muy buena práctica) y lo más importante, permitir reutilizar código (modularizar).

Si queremos diseñar una función siempre debemos tener en consideración dos cosas:

- **La declaración de la función:** Corresponde a indicarle al compilador que en algún punto del código más adelante, podría existir la utilización de una función que aún no sabemos qué hace. La declaración lleva la siguiente sintaxis: `tipo_dato_retornado nombre(argumentos);`

Por ejemplo:

```
1 int elevar_cuadrado(int);
```

- **La definición de la función:** Corresponde a indicarle al compilador qué es lo que debe realizar la función. Siguiendo el mismo ejemplo anterior, la definición de la función sería:

```
1 int elevar_cuadrado(int entrada)
2 {
3     int salida;
4     salida = entrada * entrada;
5     return salida;
6 }
```

La definición de una función debe llevar el mismo encabezado de la declaración, agregando los argumentos de entrada, seguido de las llaves {, luego todas las líneas de código que deba realizar y finalmente la llave }. El keyword `return` indica que en esta línea la función terminará, y devolverá el dato que sigue a continuación (`tipo_dato_retornado` y lo que retorna deben ser del mismo tipo). Para entender mejor esto observe cómo se llamaría esta función dentro de un programa:

```
1 int elevar_cuadrado(int);
2
3 int main()
4 {
5     int a = 5;
6     int b = elevar_cuadrado(a);
7     return 0;
8 }
9
10 int elevar_cuadrado(int entrada)
11 {
12     int salida;
13     salida = entrada * entrada;
14     return salida;
15 }
```

Como puede observar, la declaración va sobre la función `main` mientras que la definición puede ir abajo. Es por esto que una muy buena práctica que todo desarrollador debería seguir es crear dos archivos extra y agregarlos a su proyecto:

- **funciones.h (o un nombre descriptivo):** Este archivo de cabecera debe contener todas las declaraciones de funciones que creamos. Este archivo debe guardarse en el directorio `Core/Inc` del proyecto e incluirse en el archivo `main.c` mediante la directiva:

```
1 #include "funciones.h"
```

- **funciones.c (o un nombre descriptivo):** Este es un archivo fuente, al igual que `main.c`, con la diferencia de que contendrá las definiciones de las funciones a implementar en `main.c` u otro `archivo.c`. Este archivo debe guardarse en el directorio `Core/Src` del proyecto

Cree un proyecto nuevo llamado “funciones”. Configúrelo y realice las modificaciones mencionadas a las 15 líneas del código de ejemplo mostrado más arriba, para separar en 3 archivos el programa anterior. Asegúrese de poder debuggear sin problemas antes de continuar.

5.1. Paso de argumentos por valor

En este ejercicio se explorarán dos formas de manipular valores o elementos dentro de una función. Conviene distinguir dos términos: los **parámetros** son las variables declaradas en el encabezado de la función (`radio`, `number`), mientras que los **argumentos** son los valores entregados en la llamada (`radio` y `10` en `main.c`). En C el paso de argumentos es siempre *por valor*: al llamar a la función se copia el valor de cada argumento en el parámetro correspondiente, de modo que la función trabaja sobre una copia, y el resultado de su ejecución se recupera mediante el valor de retorno, si es que debe hacerlo. Observe e implemente el siguiente ejemplo agregando las líneas a sus archivos respectivos:

funciones.h

```
1 #ifndef FUNCIONES_H
2 #define FUNCIONES_H
3 #define PI 3.141592
4 float area_circulo(int radio);
5 char neg_or_pos(int number);
6 #endif
```

funciones.c

```
1 #include "funciones.h" //para poder usar PI
2 float area_circulo(int radio) // esta funcion retorna el area de un circulo
3 {
4     return PI * radio * radio;
5 }
6 char neg_or_pos(int number) // esta funcion retorna un caracter p, n o z
7 {
8     if(number < 0)
9     {
10         return 'n';
11     } else if(number > 0)
12     {
13         return 'p';
14     } else
15     {
16         return 'z';
17     }
18 }
```

main.c

```
1 #include "funciones.h"
2
3 int main() // en estos casos los argumentos se pasan por valor
4 {
5     int radio = 5;
6     float area = area_circulo(radio);
7     char pos = neg_or_pos(10);
8     return 0;
9 }
```

Corra su programa paso a paso, revise las variables en la ventana *expressions* y responda:

- ¿Qué ocurre con los parámetros dentro de las funciones? ¿Qué relación tienen con los argumentos de la llamada? Note que **radio** en **main** y **radio** en **area_circulo** son variables distintas.
- ¿Qué valor se asigna a las variables **area** y **pos** de **main** en cada llamada?
- Modifique la función **area_circulo** para que utilice la función **eleva_cuadrado** definida en el punto anterior para calcular el área.

5.2. Paso de argumentos por referencia

En C no existe el paso por referencia: todo argumento se pasa por valor. Lo que habitualmente se llama “paso por referencia” se emula pasando como argumento la *dirección* de la variable que se desea modificar. Esa dirección (un puntero) también se copia por valor en el parámetro, pero al desreferenciarla la función accede a la variable original de quien la llamó, y por lo tanto puede modificarla. A continuación se muestra un ejemplo de dicha práctica:

funciones.h

```
1 #ifndef FUNCIONES_H
2 #define FUNCIONES_H
3 int funcion_1(int, int);
4 int funcion_2(int*, int*);
5 #endif
```

funciones.c

```
1 int funcion_1(int x, int y)
2 {
3     x = y;
4     return x;
5 }
6 int funcion_2(int* x, int* y)
7 {
8     *x = *y;
9     return *x;
10 }
```

main.c

```
1 #include "funciones.h"
2 int main()
3 {
4     int c = 1, d = 2;
5     int e = 1, f = 2;
6     int res1, res2;
7
8     res1 = funcion_1(c,d);
9     res2 = funcion_2(&e,&f);
10    return 0;
11 }
```

Cree un nuevo proyecto llamado **paso_por_referencia**. Configúrelo y corra el ejemplo anterior paso a paso, revise las variables en la ventana *expressions* y responda:

- Observe los valores de las variables **c**, **d**, **e** y **f** antes y después de cada llamada. ¿Qué puede decir de **funcion_1**? ¿y de **funcion_2**?
- Implemente la función llamada **funcion_3** que recibe como parámetro un arreglo de 5 enteros y retorna la suma de sus elementos. Recuerde utilizar **funciones.c** y **funciones.h**.